INHERITANCE (PEWARIS CLASS)

IDENTITAS MAHASISWA

NAMA : Faizal Dwi Septian

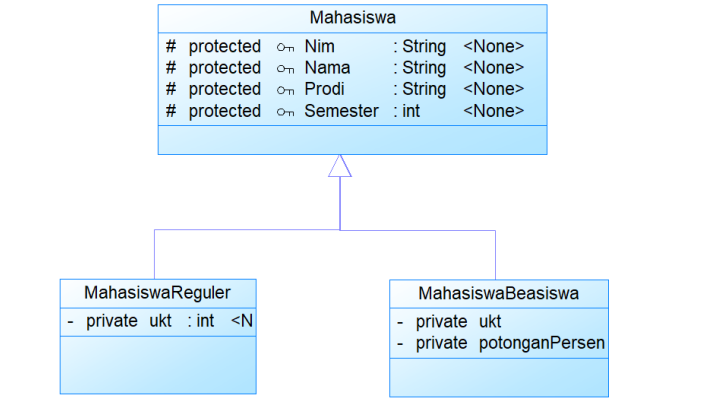
NIM : 2530521061

KELAS : 3 TI C

Tujuan Praktikum

Tujuan praktikum ini adalah untuk memahami konsep inheritance (pewarisan) dalam pemrograman berorientasi objek, mengetahui cara membuat class parent dan child, serta menerapkan pewarisan property dan method pada class. Selain itu, praktikum ini bertujuan agar mahasiswa dapat memahami penggunaan private dan protected serta menerapkan konsep inheritance dalam pembuatan program CRUD Mahasiswa.

Diagram class dan penjelasan hubungan parent–child.



Class Mahasiswa merupakan class induk (parent) yang menyimpan data umum mahasiswa seperti NIM, nama, prodi, dan semester. Data tersebut diwariskan kepada dua class turunan (child), yaitu MahasiswaReguler dan MahasiswaBeasiswa. Class MahasiswaReguler memiliki tambahan data UKT, sedangkan MahasiswaBeasiswa memiliki data UKT dan persentase potongan beasiswa. Keduanya juga memiliki method hitungBiaya untuk menghitung biaya kuliah sesuai jenis mahasiswa masing-masing. Dengan menggunakan konsep inheritance, data yang sama tidak perlu ditulis berulang kali sehingga program menjadi lebih terstruktur dan mudah dikembangkan

Potongan kode penting: extends, parent::_construct(), overriding

Extends

```
class MahasiswaReguler extends Mahasiswa {
    // Isi class MahasiswaReguler
}

class MahasiswaBeasiswa extends Mahasiswa {
    // Isi class MahasiswaBeasiswa
}
```

Parent::_construct()

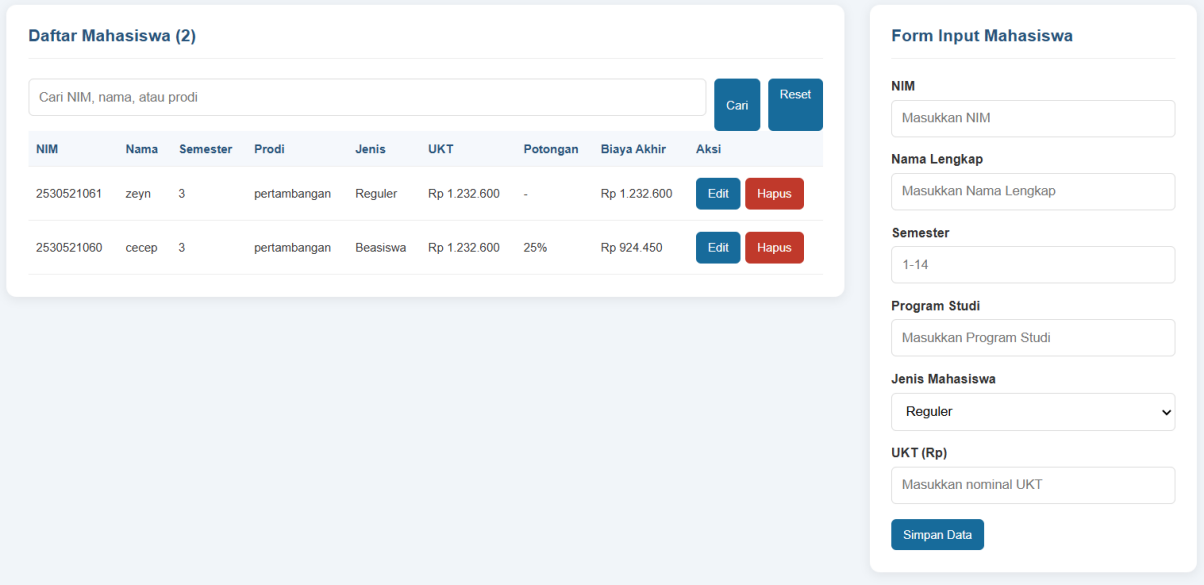
```
<?php
public function __construct($nim, $nama, $prodi, $semester, $ukt) {
    parent::__construct($nim, $nama, $prodi, $semester);
    $this->ukt = $ukt;
}

?>
```

Overriding

```
<?php
// Pada MahasiswaReguler
class MahasiswaReguler extends Mahasiswa {
    private $ukt;
    public function __construct($nim, $nama, $prodi, $semester, $ukt) {
        parent::__construct($nim, $nama, $prodi, $semester);
        $this->ukt = $ukt;
    }
    public function hitungBiaya() {
        return $this->ukt;
    }
}
// Pada MahasiswaBeasiswa
class MahasiswaBeasiswa extends Mahasiswa {
    private $ukt;
    private $potonganPersen;
    public function __construct($nim, $nama, $prodi, $semester, $ukt,
$potonganPersen) {
        parent::__construct($nim, $nama, $prodi, $semester);
        $this->ukt = $ukt;
        $this->potonganPersen = $potonganPersen;
    }
    public function hitungBiaya() {
        return intdiv($this->ukt * (100 - $this->potonganPersen), 100);
    }
}
```

Screenshot CRUD untuk Reguler dan Beasiswa.



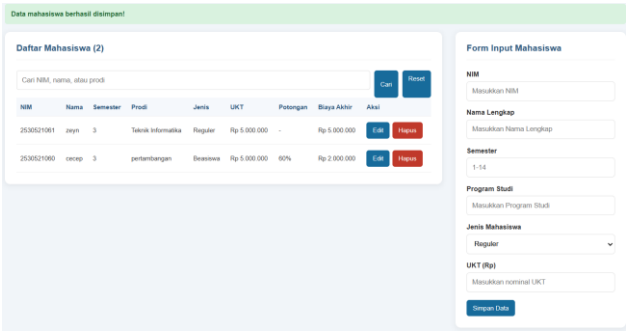
Tabel 12 hasil pengujian: input, keluaran, status sesuai/tidak.

No	Skenario	Hasil yg diharapkan	Sesuai/Tidak
1	Tambah Reguler UKT 5.000.000	Biaya akhir 5.000.000	Sesuai
2	Tambah Beasiswa UKT 5.000.000; potongan 60%	Biaya akhir 2.000.000	Sesuai
3	Nim kosong	Ditolak parent constructor	Sesuai
4	Semester 0 atau 15	Ditolak parent constructor	Sesuai
5	UKT – 100	Ditolak child constructor	Sesuai
6	Potongan 120%	Ditolak child constructor	Sesuai
7	NIM duplikat	Ditolak repository	Sesuai
8	Edit Reguler menjadi Beasiswa pada nim sama	Jenis berubah; data tetap setu record	Sesuai
9	Edit dengan NIM berbeda	Ditolak repository	Sesuai
10	Hapus satu record	Record lain tetap tersedia	Sesuai
11	Refresh browser selama session aktif	Data tetap ada	Sesuai
12	Akses $m->nim langsung	Ditolak karena private	

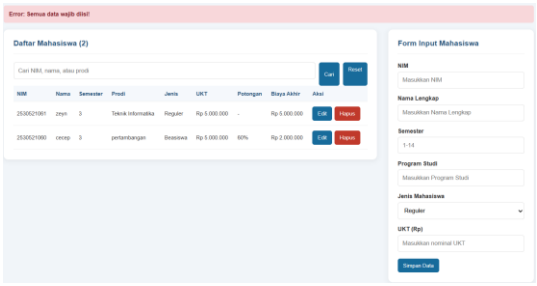
Gambar 1 Sesuai



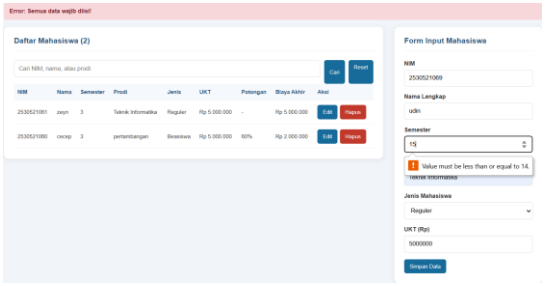
Gambar 2 Sesuai



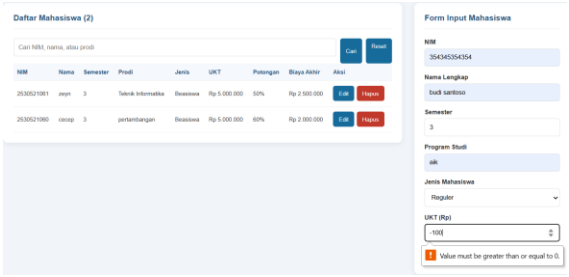
Gambar 3 Sesuai



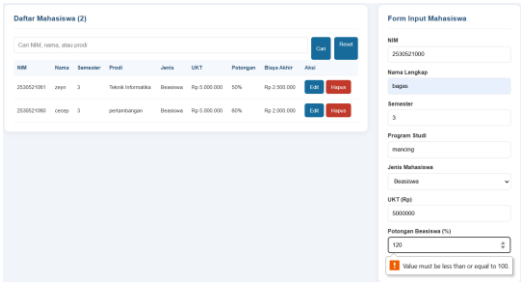
Gambar 4 Sesuai



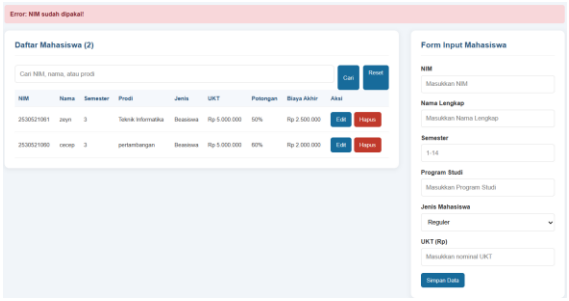
Gambar 5 Sesuai



Gambar 6 Sesuai



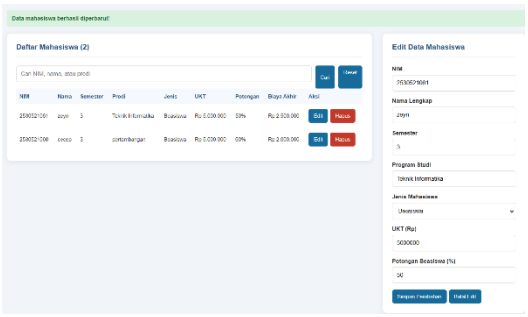
Gambar 7 Sesuai



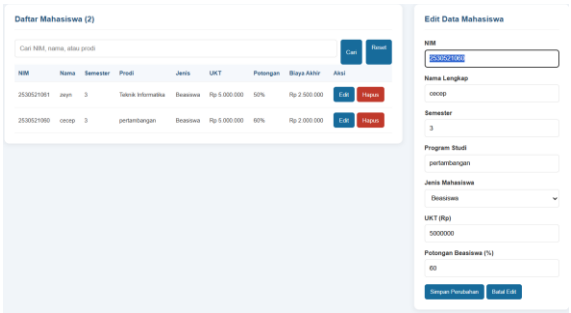
Gambar 8 Sesuai

Gambar 9 Sesuai

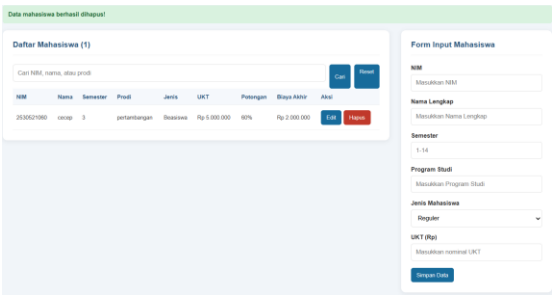
Gambar 10 Sesuai

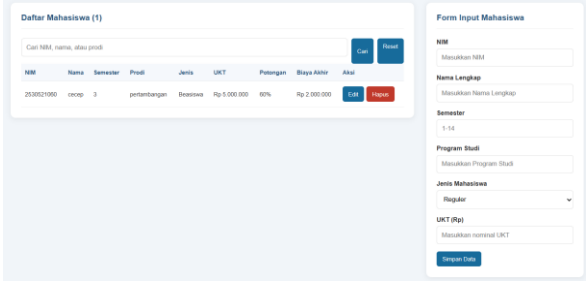


Gamabar 11 Sesuai

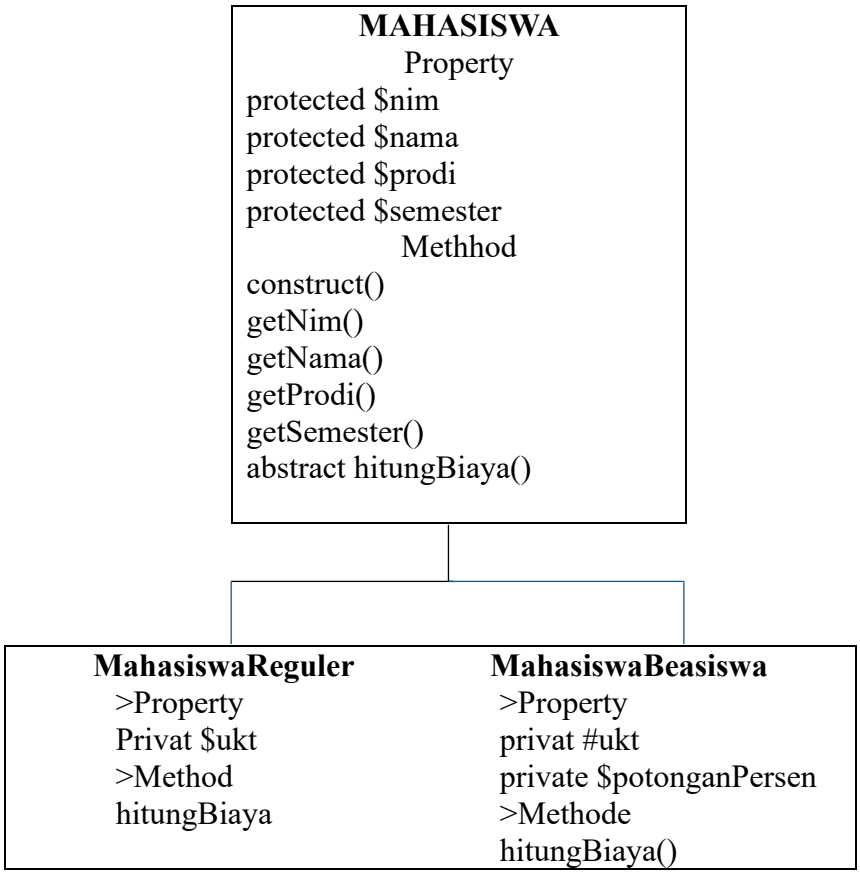


Gambar 12





Tugas 1



1. Menghindari pengulangan kode. Data yang sama seperti NIM, nama, prodi, dan semester cukup dibuat di parent class Mahasiswa, sehingga tidak perlu ditulis ulang pada setiap child class.

2. Memisahkan data sesuai jenis mahasiswa. Mahasiswa reguler memiliki data UKT, sedangkan mahasiswa beasiswa memiliki data UKT dan persentase potongan. Jadi, masing-masing data khusus ditempatkan di child class.

3. Memudahkan pengembangan program. Jika nantinya ada jenis mahasiswa baru, class baru bisa dibuat dengan mewarisi class Mahasiswa tanpa harus mengubah seluruh kode yang sudah ada.

Perbandingan private dan protected

Jenis	Private	Protected
Class sendiri	Bisa	Bisa
Child class	Tidak bisa langsung	Bisa
Class lain	Tidak bisa langsung	Tidak bisa langsung

Tugas2

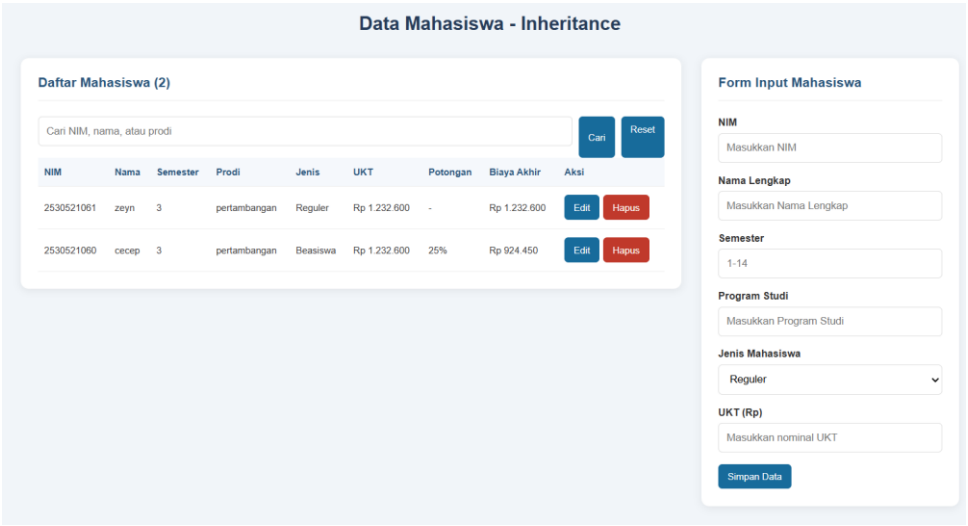
```php
<?php
// Parent Class Pegawai
class Pegawai {
    protected $id;
    protected $nama;
    public function __construct($id, $nama) {
        $this->id = $id;
        $this->nama = $nama;
    }
    public function getNama() {
        return $this->nama;
    }
}
// Child Class PegawaiTetap
class PegawaiTetap extends Pegawai {
    private $gajiPokok;
    public function __construct($id, $nama, $gajiPokok) {
        parent::__construct($id, $nama);
        if ($gajiPokok < 0) {
            throw new Exception("Gaji tidak boleh negatif!");
        }
        $this->gajiPokok = $gajiPokok;
    }
    public function hitungGaji() {
        return $this->gajiPokok;
    }
}
// Child Class PegawaiKontrak
class PegawaiKontrak extends Pegawai {
    private $upahPerJam;
    private $jamKerja;
    public function __construct($id, $nama, $upahPerJam, $jamKerja) {
        parent::__construct($id, $nama);
        if ($upahPerJam < 0 || $jamKerja < 0) {
            throw new Exception("Upah dan jam kerja tidak boleh negatif!");
        }
        $this->upahPerJam = $upahPerJam;
        $this->jamKerja = $jamKerja;
    }
    public function hitungGaji() {
        return $this->upahPerJam * $this->jamKerja;
    }
}
// Membuat dua object pegawai
try {
    $pegawai1 = new PegawaiTetap(
        "P001", "Andi", 5000000
```

```
    );
    $pegawai2 = new PegawaiKontrak(
        "P002", "Budi", 25000, 160
    );
    // Menampilkan hasil
    echo "<h3>Data Pegawai</h3>";
    echo "Nama Pegawai Tetap: " . $pegawai1->getNama() . "<br>";
    echo "Gaji Pegawai Tetap: Rp " . number_format($pegawai1->hitungGaji(), 0,
',', '.') . "<br><br>";
    echo "Nama Pegawai Kontrak: " . $pegawai2->getNama() . "<br>";
    echo "Gaji Pegawai Kontrak: Rp " . number_format($pegawai2->hitungGaji(),
0, ',', '.');
} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
}
```

Output



Tugas 3



https://github.com/zeyn-lgtm/PBO_06

Tugas 4

1. Mahasiswa – MahasiswaBeasiswa (Is-A)

MahasiswaBeasiswa merupakan salah satu jenis mahasiswa yang mendapatkan beasiswa. Karena itu, MahasiswaBeasiswa bisa mewarisi data dari class Mahasiswa, seperti NIM, nama, prodi, dan semester. Selain itu, MahasiswaBeasiswa juga memiliki data tambahan berupa potongan beasiswa.

2. Mahasiswa – Alamat (Has-A)

Mahasiswa memiliki alamat sebagai informasi tempat tinggalnya. Alamat bukan termasuk jenis mahasiswa, tetapi merupakan data yang dimiliki oleh mahasiswa. Jadi, hubungan ini lebih tepat menggunakan komposisi.

3. Mahasiswa – KartuPerpustakaan (Has-A)

Mahasiswa dapat memiliki kartu perpustakaan yang digunakan untuk meminjam buku. Kartu perpustakaan bukan bagian dari jenis mahasiswa, melainkan objek tersendiri yang dimiliki oleh mahasiswa. Jadi, hubungan ini termasuk komposisi.

4. Mahasiswa – MahasiswaReguler (Is-A)

MahasiswaReguler merupakan salah satu jenis mahasiswa yang mengikuti perkuliahan reguler. Class MahasiswaReguler dapat mewarisi data umum dari class Mahasiswa, seperti NIM, nama, prodi, dan semester. Selain itu, MahasiswaReguler memiliki data tambahan berupa UKT.

Contoh desain inheritance yang keliru dan perbaikannya

Contoh desain yang keliru adalah menjadikan class Alamat sebagai turunan dari class Mahasiswa. Hal tersebut kurang tepat karena alamat bukan jenis mahasiswa, melainkan informasi yang dimiliki mahasiswa.

Perbaikannya adalah menggunakan komposisi, yaitu class Mahasiswa memiliki objek Alamat. Dengan begitu, data mahasiswa dan alamat bisa dibuat secara terpisah dan lebih mudah dikelola.

Pertanyaan penguatan konsep

1.Mengapa MahasiswaBeasiswa disebut is-a Mahasiswa, bukan has-a Mahasiswa?

Karena MahasiswaBeasiswa merupakan salah satu jenis mahasiswa yang mendapatkan beasiswa. Jadi, MahasiswaBeasiswa bisa mewarisi data dan method dari class Mahasiswa, seperti NIM, nama, prodi, dan semester. Kalau has-a artinya memiliki sesuatu, bukan merupakan jenis dari sesuatu.

2.Apa yang terjadi jika child membuat constructor sendiri tetapi lupa memanggil parent::__construct()?

Jika child membuat constructor sendiri tetapi tidak memanggil parent::__construct(), maka constructor milik parent tidak akan dijalankan. Akibatnya, data yang seharusnya diisi oleh parent, seperti NIM dan nama, bisa tidak terisi dengan benar.

3.Mengapa child tidak boleh langsung membaca private $nim pada parent?

Karena properti yang menggunakan private hanya bisa diakses dari dalam class tempat properti tersebut dibuat. Jadi, child tidak bisa langsung membaca $nim milik parent. Untuk mengaksesnya, bisa menggunakan method getter yang sudah dibuat di parent.

4.Apa beda overriding dengan overloading yang dipelajari pada Pertemuan 4?

Overriding adalah ketika child membuat ulang method yang sudah ada di parent dengan nama yang sama, tetapi isi atau prosesnya berbeda. Sedangkan overloading adalah penggunaan nama method yang sama dengan parameter yang berbeda. Jadi, keduanya sama-sama berkaitan dengan penggunaan method, tetapi caranya berbeda.

5.Mengapa method hitungBiaya() dapat dipanggil dari variabel bertipe Mahasiswa?

Karena class Mahasiswa memiliki method abstrak hitungBiaya() yang harus dibuat ulang oleh class turunannya. Jadi, meskipun variabelnya bertipe Mahasiswa, objek yang digunakan bisa menjalankan method hitungBiaya() sesuai jenis mahasiswa tersebut, baik Reguler maupun Beasiswa.

6.Apakah use TraitX sama dengan extends ParentX? Jelaskan.

Tidak sama. extends ParentX digunakan untuk mewarisi properti dan method dari class parent. Sedangkan use TraitX digunakan untuk menggunakan kumpulan method dari sebuah trait ke dalam class. Jadi, trait bisa digunakan untuk berbagi kode tanpa harus membuat hubungan pewarisan antarclass.

Kesimpulan

Setelah melakukan praktikum tentang inheritance, saya jadi lebih memahami bagaimana sebuah class dapat mewarisi data dan method dari class lain. Saya juga belajar bahwa inheritance bisa membuat kode menjadi lebih sederhana karena tidak perlu menuliskan data yang sama berulang kali. Selain itu, saya memahami perbedaan antara parent class dan child class, penggunaan constructor, serta penerapan method overriding. Melalui contoh Mahasiswa Reguler dan Mahasiswa Beasiswa, saya jadi lebih mengerti bagaimana inheritance dapat digunakan dalam pembuatan program.

LINK GITHUB

https://github.com/zeyn-lgtm/PBO_06